CS2201 Computer Organization & Architecture

L9: Instruction Pipeline Hazards



John Jose

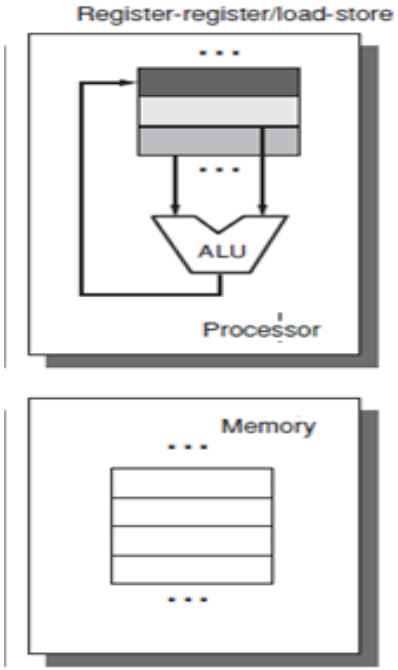
Associate Professor

Department of Computer Science & Engineering

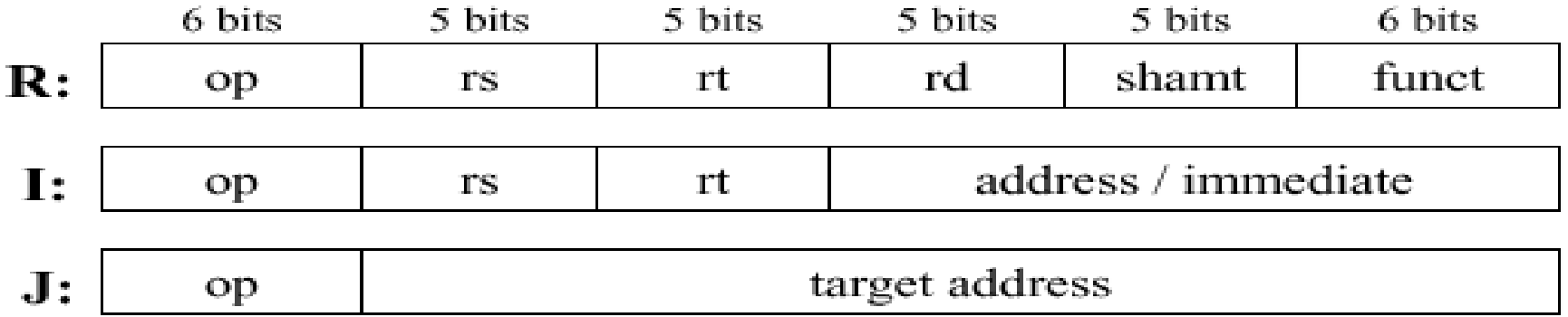
Indian Institute of Technology Guwahati

Introduction to MIPS

- Microprocessor without Interlocked Pipelined Stages
- RISC- Load store architecture
- 32 registers (32 bit each)
- Uniform length instructions

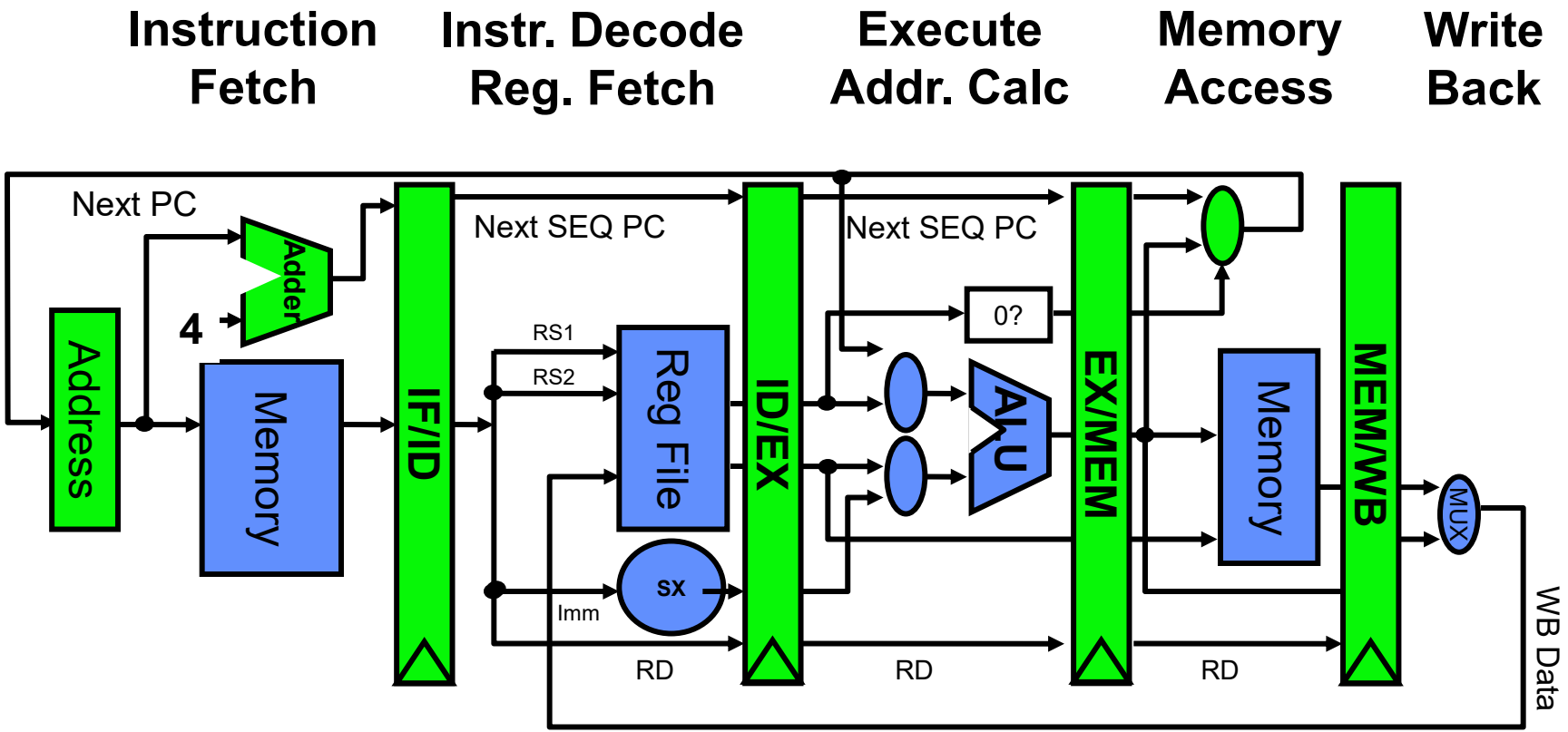


Introduction to MIPS



op: basic operation of the instruction (opcode)
rs: first source operand register
rt: second source operand register
rd: destination operand register
shamt: shift amount
funct: selects the specific variant of the opcode (function code)
address: offset for load/store instructions (+/-2^{15})
immediate: constants for immediate instructions

5 Stage RISC Pipeline



5 Steps of RISC Data path

- Each instruction can take at most 5 clock cycles
- **Instruction fetch cycle (IF)**
  - Based on PC value, fetch the instruction from memory
  - Update PC
- **Instruction decode/register fetch cycle (ID)**
  - Decode the instruction + register read operation
  - Equality check of registers
  - Computation of branch target address if any

5 Steps of MIPS Data path

- **Execution/Effective address cycle (EX)**
  - Memory reference: Calculate the effective address
  - Register-register ALU instruction
- **Memory access cycle (MEM)**
  - Load instruction: Read from memory using effective address
  - Store instruction: Write the data in the register to memory using effective address

5 Steps of MIPS Data path

- **Write-back cycle (WB)**
  - Register-register ALU instruction or load instruction
  - Write the result into the register file

Pipelining Issues

- **Ideal Case: Uniform sub-computations**
  - The computation to be performed can be evenly partitioned into uniform-latency sub-computations
- **Reality: Internal fragmentation**
  - Not all pipeline stages may have the uniform latencies
- **Impact of ISA**
  - Memory access is a critical sub-computation
  - Memory addressing modes should be minimized
  - Fast cache memories should be employed

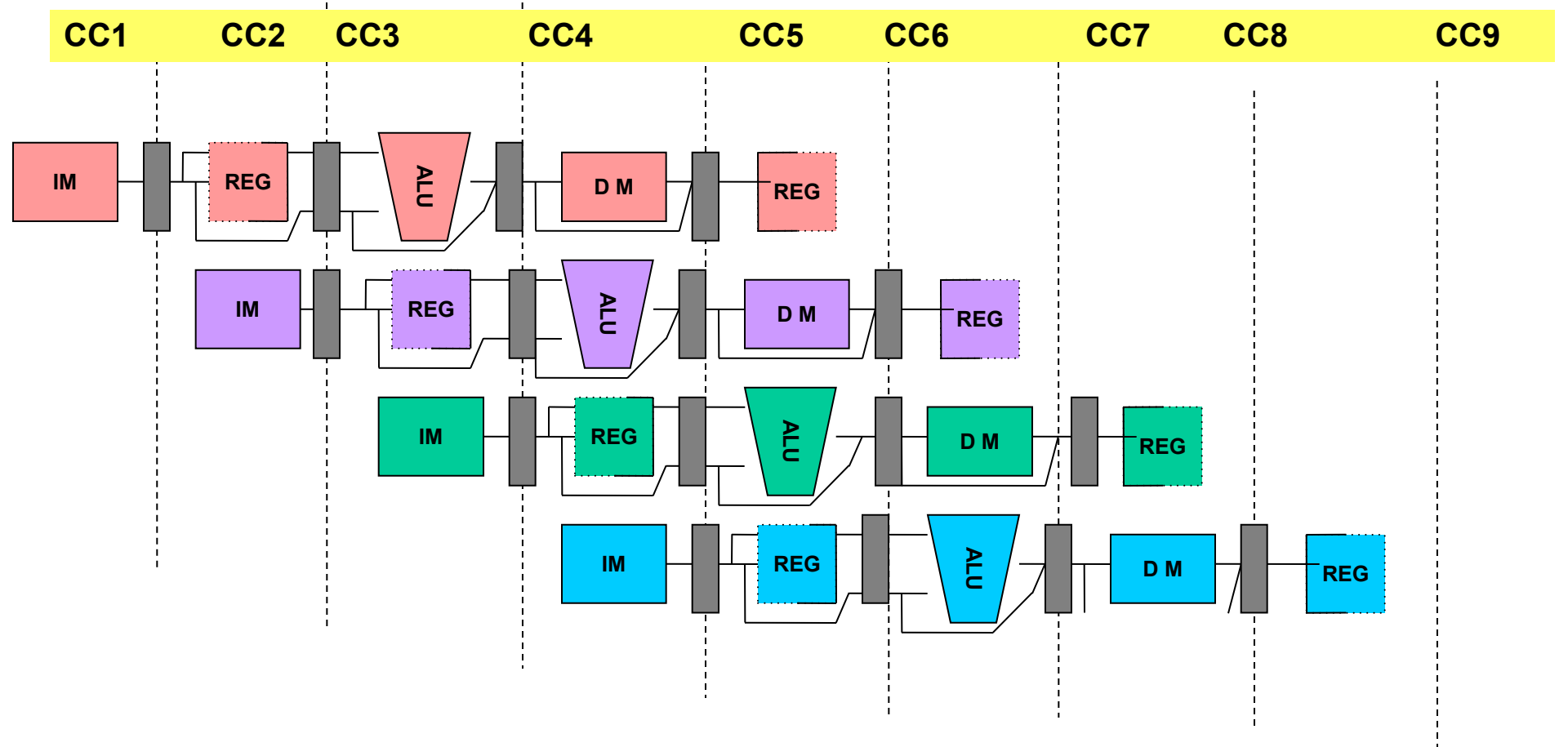
Pipelining Issues

- **Ideal Case : Identical computations**
  - The same computation is to be performed repeatedly on a large number of input data sets
- **Reality: External fragmentation**
  - Some pipeline stages may not be used
- **Impact of ISA**
  - Reduce the complexity and diversity of the instruction types
  - RISC architectures use uniform stage simple instructions

Pipelining Issues

- **Ideal Case : Independent computations**
  - All the instructions are mutually independent
- **Reality: Pipeline stalls – cannot proceed.**
  - A later computation may require the result of an earlier computation
- **Impact of ISA**
  - Reduce Memory addressing modes - dependency detection
  - Use register addressing mode - easy dependencies check

Visualizing Pipelining

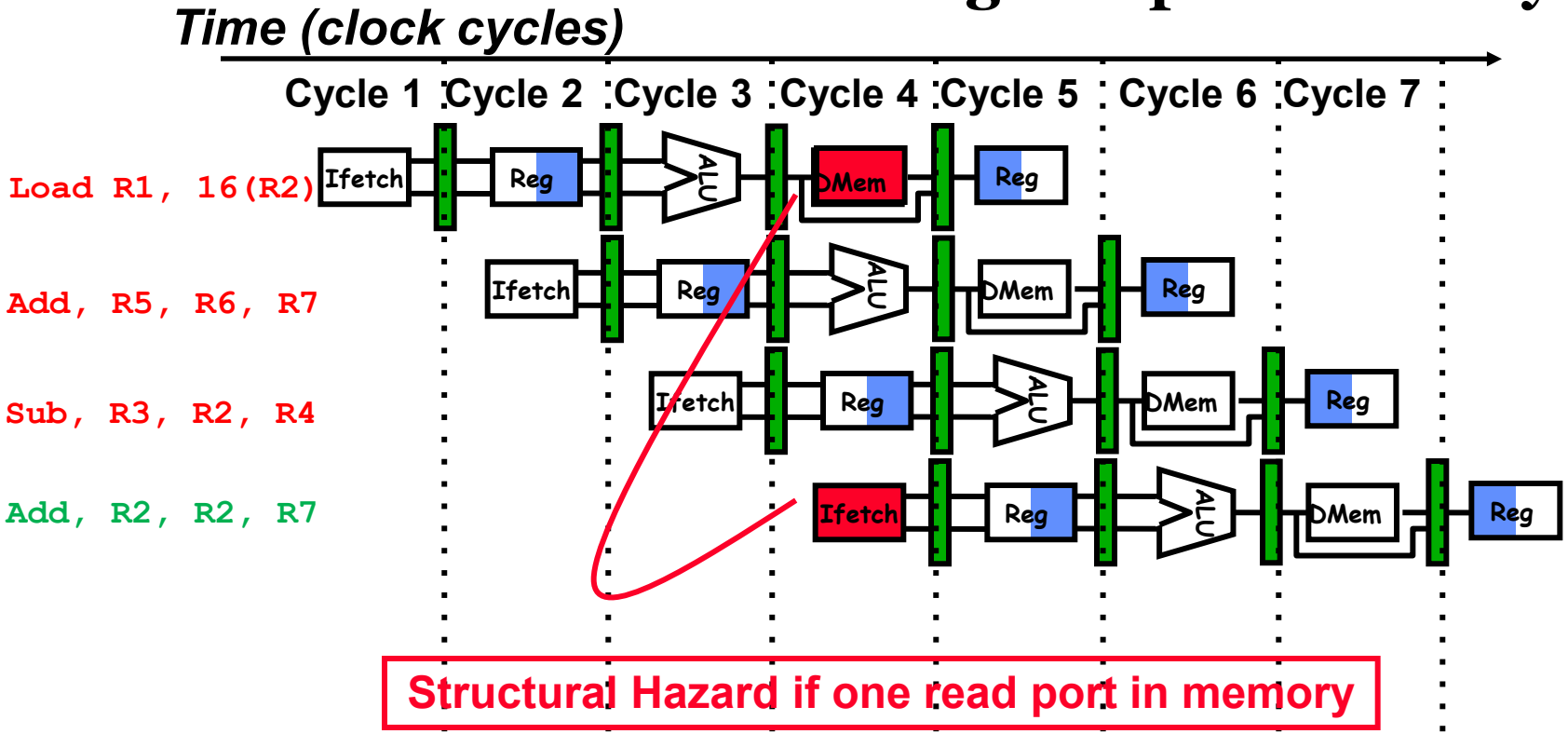


Limits to pipelining

- **Hazards:** circumstances that would cause incorrect execution if next instruction is fetched and executed
  - **Structural hazards:** Different instructions, at different stages, in the pipeline want to use the same hardware resource
  - **Data hazards:** An instruction in the pipeline requires data to be computed by a previous instruction still in the pipeline
  - **Control hazards:** Succeeding instruction, to put into pipeline, depends on the outcome of a previous branch instruction, already in pipeline

Structural Hazard

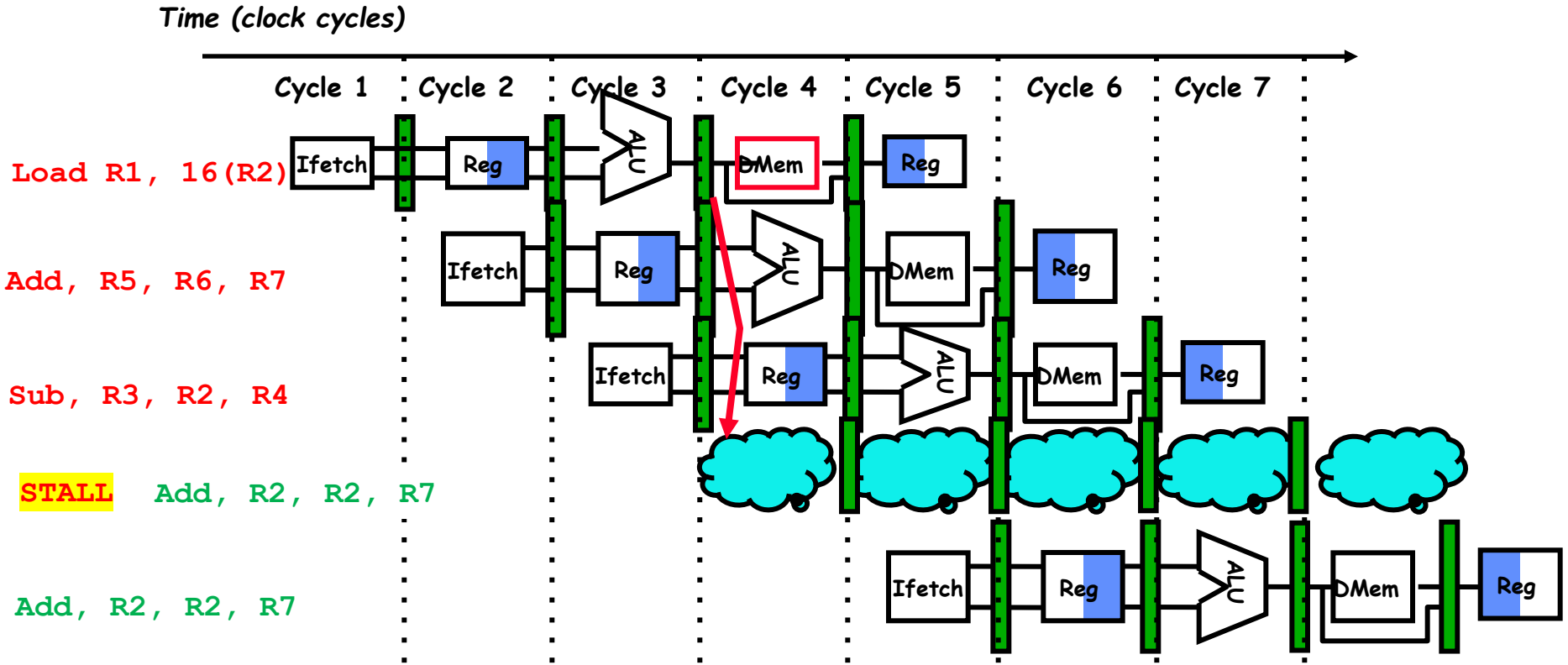
Eg: Uniport Memory



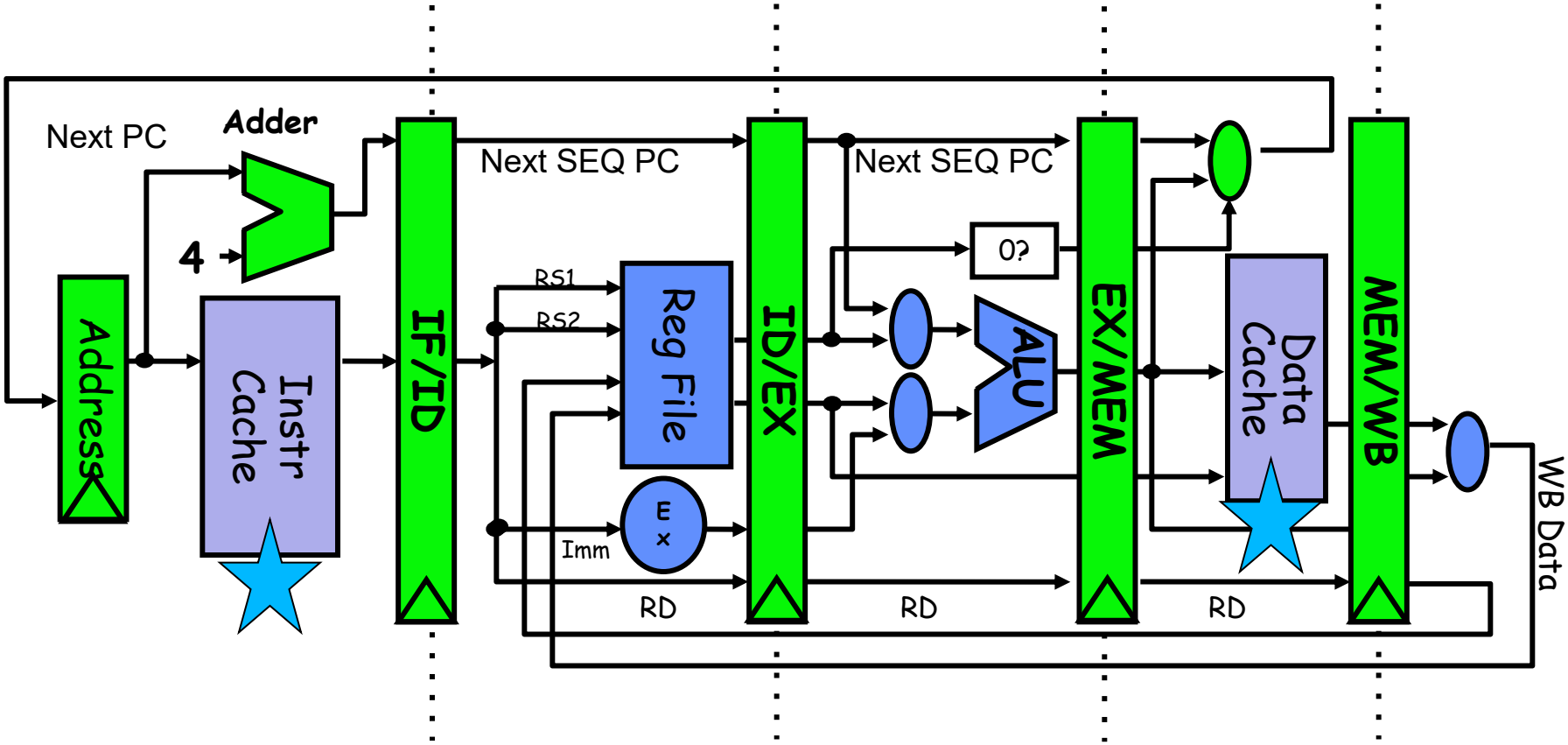
Resolving Structural Hazard

- Eliminate the use same hardware for two different things at the same time
- **Solution 1: Wait**
  - must detect the hazard
  - must have mechanism to stall
- **Solution 2: Duplicate hardware**
  - Multiple such units will help both instruction to progress

Detecting & Resolving Structural Hazard



Eliminating Structural Hazards at Design





johnjose@iitg.ac.in

http://www.iitg.ac.in/johnjose/

https://www.linkedin.com/in/john-jose-bb8431106/